VLSI INTERNSHIP PROJECT REPORT

SAN Web Technology • 2026

Digital Logic • Verilog RTL • Testbenches • Simulation • Git • Synthesis • Physical Design

Includes all nine tasks across Beginner, Intermediate and Advanced levels, with source code, testbenches, README files, scripts/configuration examples and waveform references.

Prepared from the supplied SAN Web Technology VLSI Internship Task Guide.

Task Coverage

All nine tasks in the guide are represented in the project pack. See the task-specific folders, README files and source appendices.

Level	Task	Included deliverables
1	1. Logic Gates & RTL Simulation	RTL, testbench, README, waveform reference
1	2. RTL & Testbench Practice	Counter/register RTL, testbench, command instructions, README
1	3. Git-to-RTL Verification Flow	Edge counter RTL, testbench, Git workflow, README
2	1. Parameterized RTL Module	Parameterized counter, testbench, README
2	2. RTL-to-Synthesis Flow	ALU RTL, testbench, Yosys script, README
2	3. FSM Verification & Data Flow	FSM RTL, testbench, README, waveform reference
3	1. Automated RTL Verification	GitHub Actions workflow YAML, README
3	2. RTL-to-GDSII Mini Flow	Flow config, SDC template, stage checklist
3	3. Timing, Area & Power Review	Review template and checklist

Waveform note: diagrams in this report are expected/reference timing diagrams, not actual simulator captures. The environment used to build this pack did not have Icarus Verilog, GTKWave, Yosys or OpenROAD installed. Run the included testbenches to generate actual VCDs, and add real GTKWave screenshots and tool reports before submission.

Run Commands and Expected Behavior

Level 1 Task 1

```
cd level1/task1_logic_gates
iverilog -g2012 -o simv rtl/logic_gates.v tb/tb_logic_gates.v
vvp simv
gtkwave task1_logic_gates.vcd
```

Expected / evidence: AND/OR/XOR/NAND truth tables; mux selects a for sel=0 and b for sel=1.

Level 1 Task 2

```
cd level1/task2_rtl_testbench
mkdir -p build
iverilog -g2012 -Wall -o build/task2_sim rtl/counter_register.v tb/tb_counter_register.v
vvp build/task2_sim | tee command_log.txt
gtkwave task2_counter_register.vcd
```

Expected / evidence: Reset clears both registers; enable increments count and captures input; disabled values hold.

Level 1 Task 3

```
cd level1/task3_git_verification
iverilog -g2012 -o edge_sim rtl/edge_counter.v tb/tb_edge_counter.v
vvp edge_sim
gtkwave task3_edge_counter.vcd
```

Expected / evidence: Pulse increments the edge counter; record actual Git commit IDs and passing outputs.

Level 2 Task 1

```
cd level2/task1_parameterized_rtl
iverilog -g2012 -Wall -o counter_sim rtl/parameterized_counter.v tb/tb_parameterized_counter.v
vvp counter_sim
gtkwave parameterized_counter.vcd
```

Expected / evidence: Verify parameterized widths/reset values, enable hold and wraparound.

Level 2 Task 2

```
cd level2/task2_rtl_to_synthesis
mkdir -p build
iverilog -g2012 -Wall -o alu_sim rtl/alu4.v tb/tb_alu4.v
vvp alu_sim
yosys -s synth/synth.ys | tee build/yosys_report.txt
```

Expected / evidence: ALU op: 00 add, 01 subtract, 10 AND, 11 XOR. Use actual Yosys report for metrics.

Level 2 Task 3

```
cd level2/task3_fsm_verification
iverilog -g2012 -Wall -o fsm_sim rtl/sequence_controller.v tb/tb_sequence_controller.v
vvp fsm_sim
gtkwave sequence_controller.vcd
```

Expected / evidence: Expected path IDLE → LOAD → STEP → FINISH → IDLE; busy high in LOAD/STEP; done high in FINISH.

Level 3 Task 1

```
Copy the workflow YAML to the root of the GitHub repo with matching RTL/testbench paths; commit and push; inspect Actions and artifacts
```

Expected / evidence: Capture an actual successful workflow run; never add secrets to source.

Level 3 Task 2

```
Run the provided config and SDC with your installed OpenLane/OpenROAD version after checking PDK, paths and version-specific syntax.
```

Expected / evidence: Retain actual synthesis, floorplan, placement, routing and GDSII evidence only when generated.

Level 3 Task 3

```
Fill level3/task3_timing_area_power/implementation_review.md from real reports.
```

Expected / evidence: Do not invent area, slack or power values; document one measured risk and a tested improvement.

Appendix A — Source Code, Configuration and README Files

The following appendix embeds the actual text of the project files so the report contains the code and documentation, not only file names. The same files are also organized in the downloadable ZIP pack.

README.md

```
# SAN Web Technology — VLSI Internship Submission Pack (2026)

This project pack maps to all nine tasks listed in the supplied internship guide. It includes RTL,
    testbenches, simulation commands, synthesis/flow configuration examples, and README files for
    each task.

## Task coverage
| Level | Task | Included |
|---|---|---|
| 1 | 1. Logic Gates & RTL Simulation | RTL, testbench, expected waveform reference, README |
| 1 | 2. RTL & Testbench Practice | Counter/register RTL, testbench, CLI commands, README |
| 1 | 3. Git-to-RTL Verification Flow | Edge-counter RTL, testbench, Git workflow, README |
| 2 | 1. Parameterized RTL Module | Parameterized counter RTL, testbench, README |
| 2 | 2. RTL-to-Synthesis Flow | 4-bit ALU, testbench, Yosys script, README |
| 2 | 3. FSM Verification & Data Flow | Sequence controller, testbench, state flow notes, README |
| 3 | 1. Automated RTL Verification | GitHub Actions YAML, setup instructions, README |
| 3 | 2. RTL-to-GDSII Mini Flow | OpenLane-style config template, SDC, stage checklist, README |
| 3 | 3. Timing, Area & Power Review | Measurement template and checklist, README |

## Tool status and evidence honesty
The current environment used to assemble this pack did not have Icarus Verilog, GTKWave, Yosys or
    OpenROAD installed. Therefore the waveform images in the report are clearly marked
    **expected/reference timing diagrams**, not screenshots captured from a simulator. RTL and
    scripts are supplied so you can run them on your configured machine. Replace/add actual
    simulator screenshots, generated VCDs, synthesis reports, GitHub Actions evidence and physical-
    design reports after running the tools.

## Quick start (Icarus Verilog + GTKWave)
Example for Level 1 Task 2:
```sh
cd level1/task2_rtl_testbench
mkdir -p build
iverilog -g2012 -Wall -o build/task2_sim rtl/counter_register.v tb/tb_counter_register.v
vvp build/task2_sim | tee command_log.txt
gtkwave task2_counter_register.vcd
```

Submission items still requiring your own run/account
- Push the repository and add its actual URL.
- Generate actual waveforms and capture GTKWave screenshots after running testbenches.
- Run Yosys/OpenLane/OpenROAD tasks in a correctly configured environment and retain real
    reports/screenshots.
- Add your real LinkedIn completion/project post and tag the official SAN Web Technology handle
    provided by the company.
- Submit/deposit through the method provided by SAN Web Technology.
- Do not publish secrets, licensed PDKs, credentials or private IP.
```

level1/task1_logic_gates/README.md

```
# Level 1 — Task 1: Logic Gates & RTL Simulation

## Goal
Implement AND, OR, XOR, NAND and a 2:1 multiplexer, test every input combination, and inspect a waveform.

## Files
- `rtl/logic_gates.v` — combinational RTL.
- `tb/tb_logic_gates.v` — testbench and VCD dump.
- `waveforms/task1_logic_gates_reference.png` — expected timing reference, not a simulator capture.

## Run with Icarus Verilog
From this task directory:
```sh
mkdir -p build waveforms
iverilog -g2012 -o build/simv rtl/logic_gates.v tb/tb_logic_gates.v
vvp build/simv
gtkwave task1_logic_gates.vcd
```
Expected: all four gate outputs follow their truth tables; mux output is `a` when `sel=0` and `b`
    when `sel=1`.
```

level1/task1_logic_gates/rtl/logic_gates.v

```
module logic_gates(
    input wire a, b,
    output wire y_and, y_or, y_xor, y_nand
);
    assign y_and  = a & b;
    assign y_or   = a | b;
    assign y_xor  = a ^ b;
    assign y_nand = ~(a & b);
endmodule

module mux2to1(
    input wire a, b, sel,
```

```
    output wire y
);
    assign y = sel ? b : a;
endmodule
```

level1/task1_logic_gates/tb/tb_logic_gates.v

```
`timescale 1ns/1ps
module tb_logic_gates;
    reg a, b, sel;
    wire y_and, y_or, y_xor, y_nand, y_mux;
    integer i;

    logic_gates dut(.a(a), .b(b), .y_and(y_and), .y_or(y_or),
                    .y_xor(y_xor), .y_nand(y_nand));
    mux2to1 mux_dut(.a(a), .b(b), .sel(sel), .y(y_mux));

    initial begin
        $dumpfile("task1_logic_gates.vcd");
        $dumpvars(0, tb_logic_gates);
        $display("time a b sel | AND OR XOR NAND MUX");
        for (i = 0; i < 8; i = i + 1) begin
            {a,b,sel} = i[2:0];
            #10;
            $display("%0t  %b %b  %b  |  %b   %b   %b   %b   %b",
                     $time,a,b,sel,y_and,y_or,y_xor,y_nand,y_mux);
        end
        $finish;
    end
endmodule
```

level1/task2_rtl_testbench/README.md

```
# Level 1 — Task 2: RTL & Testbench Practice

## Goal
Verify a clocked counter and register block under reset, enable, normal operation and
    boundary/wraparound conditions.

## Run
```sh
mkdir -p build
iverilog -g2012 -Wall -o build/task2_sim rtl/counter_register.v tb/tb_counter_register.v
vvp build/task2_sim | tee command_log.txt
gtkwave task2_counter_register.vcd
```
The testbench creates `task2_counter_register.vcd`. Check that reset clears both outputs, `en=1`
    increments the counter and captures `data_in`, and `en=0` holds the prior values.

Important commands (short explanation)
1. `iverilog -g2012 ...` compiles RTL and testbench into a simulation executable.
2. `-Wall` requests compiler warnings.
3. `vvp build/task2_sim` runs the compiled simulation.
4. `tee command_log.txt` displays output and saves the command log.
5. `gtkwave task2_counter_register.vcd` opens the waveform viewer.
```

level1/task2_rtl_testbench/rtl/counter_register.v

```
module counter_register #(
    parameter WIDTH = 4
)(
    input wire clk,
    input wire rst_n,
    input wire en,
    input wire [WIDTH-1:0] data_in,
    output reg [WIDTH-1:0] count,
    output reg [WIDTH-1:0] data_out
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            count   <= {WIDTH{1'b0}};
            data_out <= {WIDTH{1'b0}};
        end else if (en) begin
            count   <= count + 1'b1;
            data_out <= data_in;
        end
    end
endmodule
```

level1/task2_rtl_testbench/tb/tb_counter_register.v

```
`timescale 1ns/1ps
module tb_counter_register;
    reg clk, rst_n, en;
    reg [3:0] data_in;
    wire [3:0] count, data_out;

    counter_register #(.WIDTH(4)) dut(
        .clk(clk), .rst_n(rst_n), .en(en), .data_in(data_in),
        .count(count), .data_out(data_out)
```

```
    );

    always #5 clk = ~clk;

    initial begin
        $dumpfile("task2_counter_register.vcd");
        $dumpvars(0, tb_counter_register);
        clk = 0; rst_n = 0; en = 0; data_in = 4'h0;
        #12; rst_n = 1;
        @(negedge clk); en = 1; data_in = 4'h3;
        @(negedge clk); data_in = 4'h7;
        @(negedge clk); en = 0; data_in = 4'hA;
        @(negedge clk); en = 1; data_in = 4'hF;
        @(negedge clk); data_in = 4'h1;
        @(negedge clk); rst_n = 0;
        @(negedge clk); rst_n = 1; en = 1; data_in = 4'h5;
        repeat (3) @(negedge clk);
        $finish;
    end
endmodule
```

level1/task3_git_verification/README.md

```
# Level 1 — Task 3: Git-to-RTL Verification Flow

## Goal
Track RTL changes in Git and rerun verification after each meaningful change.

## Run simulation
```sh
iverilog -g2012 -o edge_sim rtl/edge_counter.v tb/tb_edge_counter.v
vvp edge_sim
gtkwave task3_edge_counter.vcd
```

Suggested Git workflow
```sh
git init
git add .
git commit -m "Add edge counter RTL and testbench"
# Make a meaningful RTL or testbench change, then:
iverilog -g2012 -o edge_sim rtl/edge_counter.v tb/tb_edge_counter.v
vvp edge_sim
git add .
git commit -m "Verify edge counter boundary behavior"
```
Record the commit ID and attach the passing simulation output/waveform in your final repository. Do
    not claim a test passed until you run it locally.
```

level1/task3_git_verification/rtl/edge_counter.v

```
module edge_counter(
    input wire clk, rst_n, pulse,
    output reg [3:0] count
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            count <= 4'd0;
        else if (pulse)
            count <= count + 1'b1;
    end
endmodule
```

level1/task3_git_verification/tb/tb_edge_counter.v

```
`timescale 1ns/1ps
module tb_edge_counter;
    reg clk, rst_n, pulse;
    wire [3:0] count;
    edge_counter dut(.clk(clk), .rst_n(rst_n), .pulse(pulse), .count(count));
    always #5 clk = ~clk;
    initial begin
        $dumpfile("task3_edge_counter.vcd"); $dumpvars(0, tb_edge_counter);
        clk=0; rst_n=0; pulse=0;
        #12 rst_n=1;
        @(negedge clk) pulse=1;
        repeat(4) @(negedge clk);
        pulse=0;
        repeat(2) @(negedge clk);
        pulse=1;
        repeat(2) @(negedge clk);
        rst_n=0;
        @(negedge clk) rst_n=1; pulse=0;
        repeat(2) @(negedge clk);
        $finish;
    end
endmodule
```

level2/task1_parameterized_rtl/README.md

```
# Level 2 — Task 1: Parameterized RTL Module

A reusable width-configurable counter is instantiated at 4 bits and 8 bits. The 8-bit instance
    demonstrates a nonzero reset value.

```sh
iverilog -g2012 -Wall -o counter_sim rtl/parameterized_counter.v tb/tb_parameterized_counter.v
vvp counter_sim
gtkwave parameterized_counter.vcd
```
Check reset values, enable hold behavior and wraparound for the 4-bit instance. Review warnings
    before submission.
```

level2/task1_parameterized_rtl/rtl/parameterized_counter.v

```
module parameterized_counter #(
    parameter WIDTH = 8,
    parameter [WIDTH-1:0] RESET_VALUE = {WIDTH{1'b0}}
)(
    input wire clk, rst_n, en,
    output reg [WIDTH-1:0] count
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            count <= RESET_VALUE;
        else if (en)
            count <= count + {{(WIDTH-1){1'b0}}, 1'b1};
    end
endmodule
```

level2/task1_parameterized_rtl/tb/tb_parameterized_counter.v

```
`timescale 1ns/1ps
module tb_parameterized_counter;
    reg clk, rst_n, en;
    wire [3:0] count4;
    wire [7:0] count8;
    parameterized_counter #(.WIDTH(4), .RESET_VALUE(4'd0)) c4
        (.clk(clk), .rst_n(rst_n), .en(en), .count(count4));
    parameterized_counter #(.WIDTH(8), .RESET_VALUE(8'd10)) c8
        (.clk(clk), .rst_n(rst_n), .en(en), .count(count8));
    always #5 clk=~clk;
    initial begin
        $dumpfile("parameterized_counter.vcd"); $dumpvars(0,tb_parameterized_counter);
        clk=0; rst_n=0; en=0; #12; rst_n=1; en=1;
        repeat(18) @(negedge clk);
        en=0; repeat(2) @(negedge clk);
        $finish;
    end
endmodule
```

level2/task2_rtl_to_synthesis/README.md

```
# Level 2 — Task 2: RTL-to-Synthesis Flow

## Simulate
```sh
iverilog -g2012 -Wall -o alu_sim rtl/alu4.v tb/tb_alu4.v
vvp alu_sim
gtkwave alu4.vcd
```

Synthesize with Yosys
From this task directory:
```sh
mkdir -p build
yosys -s synth/synth.ys | tee build/yosys_report.txt
```
The script checks hierarchy, lowers procedural logic, optimizes, synthesizes, prints cell statistics
     and writes `build/alu4_netlist.v`. Review the actual Yosys report and retain it in the
    submission; no area/timing numbers are pre-filled because they depend on the installed Yosys
    version and target flow.
```

level2/task2_rtl_to_synthesis/rtl/alu4.v

```
module alu4(
    input wire [3:0] a, b,
    input wire [1:0] op,
    output reg [3:0] y,
    output reg carry
);
    always @* begin
        carry = 1'b0;
        case (op)
            2'b00: begin {carry,y} = {1'b0,a} + {1'b0,b}; end
            2'b01: begin y = a - b; carry = (a >= b); end
            2'b10: begin y = a & b; end
            2'b11: begin y = a ^ b; end
            default: begin y = 4'b0; carry = 1'b0; end
        endcase
```

```
    end
endmodule
```

level2/task2_rtl_to_synthesis/synth/synth.ys

```
read_verilog rtl/alu4.v
hierarchy -check -top alu4
proc
opt
check
synth -top alu4
stat
write_verilog -noattr build/alu4_netlist.v
```

level2/task2_rtl_to_synthesis/tb/tb_alu4.v

```
`timescale 1ns/1ps
module tb_alu4;
    reg [3:0] a,b; reg [1:0] op;
    wire [3:0] y; wire carry;
    alu4 dut(.a(a),.b(b),.op(op),.y(y),.carry(carry));
    integer i;
    initial begin
        $dumpfile("alu4.vcd"); $dumpvars(0,tb_alu4);
        a=4'd9; b=4'd6;
        for(i=0;i<4;i=i+1) begin op=i[1:0]; #10; $display("op=%b a=%h b=%h y=%h
    carry=%b",op,a,b,y,carry); end
        a=4'hF; b=4'h1; op=0; #10;
        $finish;
    end
endmodule
```

level2/task3_fsm_verification/README.md

```
# Level 2 — Task 3: FSM Verification & Data Flow

The controller uses IDLE → LOAD → STEP → FINISH → IDLE. `start` initiates a transaction from IDLE.
    `busy`, `done` and `data` expose the state-dependent behavior.

```sh
iverilog -g2012 -Wall -o fsm_sim rtl/sequence_controller.v tb/tb_sequence_controller.v
vvp fsm_sim
gtkwave sequence_controller.vcd
```
Verify reset returns the controller to IDLE, all four legal states occur, and `done` is asserted in
    FINISH. The supplied testbench is a basic directed test; extend it with assertions or self-
    checking comparisons before claiming exhaustive verification.
```

level2/task3_fsm_verification/rtl/sequence_controller.v

```
module sequence_controller(
    input wire clk, rst_n, start,
    output reg busy,
    output reg done,
    output reg [1:0] data
);
    localparam IDLE=2'd0, LOAD=2'd1, STEP=2'd2, FINISH=2'd3;
    reg [1:0] state, next_state;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) state <= IDLE;
        else state <= next_state;
    end

    always @* begin
        next_state = state;
        case(state)
            IDLE:   if(start) next_state=LOAD;
            LOAD:   next_state=STEP;
            STEP:   next_state=FINISH;
            FINISH: next_state=IDLE;
            default: next_state=IDLE;
        endcase
    end

    always @* begin
        busy=1'b0; done=1'b0; data=2'b00;
        case(state)
            IDLE:   begin busy=1'b0; data=2'b00; end
            LOAD:   begin busy=1'b1; data=2'b01; end
            STEP:   begin busy=1'b1; data=2'b10; end
            FINISH: begin busy=1'b0; done=1'b1; data=2'b11; end
            default: begin busy=1'b0; done=1'b0; data=2'b00; end
        endcase
    end
endmodule
```

level2/task3_fsm_verification/tb/tb_sequence_controller.v

```
`timescale 1ns/1ps
module tb_sequence_controller;
    reg clk,rst_n,start;
    wire busy,done; wire [1:0] data;
    sequence_controller dut(.clk(clk),.rst_n(rst_n),.start(start),.busy(busy),.done(done),.data(data));
    always #5 clk=~clk;
    initial begin
        $dumpfile("sequence_controller.vcd"); $dumpvars(0,tb_sequence_controller);
        clk=0; rst_n=0; start=0; #12; rst_n=1;
        @(negedge clk) start=1;
        @(negedge clk) start=0;
        repeat(6) @(negedge clk);
        rst_n=0; repeat(2) @(negedge clk); rst_n=1;
        repeat(2) @(negedge clk);
        $finish;
    end
endmodule
```

level3/task1_automated_verification/.github/workflows/rtl-verify.yml

```
name: RTL Verification
on:
  push:
  pull_request:
jobs:
  simulate:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Install Icarus Verilog
        run: |
          sudo apt-get update
          sudo apt-get install -y iverilog
      - name: Compile and simulate
        run: |
          mkdir -p build
          iverilog -g2012 -Wall -o build/task1_sim rtl/counter_register.v tb/tb_counter_register.v
          vvp build/task1_sim | tee build/simulation.log
      - name: Upload simulation log and VCD
        uses: actions/upload-artifact@v4
        with:
          name: rtl-simulation-evidence
          path: |
            build/simulation.log
            task2_counter_register.vcd
```

level3/task1_automated_verification/README.md

```
# Level 3 — Task 1: Automated RTL Verification

This task provides a GitHub Actions workflow that compiles and simulates the Level 1
    counter/register example on pushes and pull requests.

## Setup
Copy this `.github/workflows/rtl-verify.yml` into the root of your GitHub repository, and copy the
    matching `rtl/counter_register.v` and `tb/tb_counter_register.v` files from Level 1 Task 2 into
    the same repository paths. Then push a commit and inspect the Actions run.

## Deliverables
- Repository URL: add your actual URL after publishing.
- Screenshot: capture a successful Actions run after it has actually executed.
- Artifact: download and retain the log/VCD from the workflow.
Never add tokens or credentials to YAML or source control.
```

level3/task2_rtl_to_gdsii/README.md

```
# Level 3 — Task 2: RTL-to-GDSII Mini Flow

## Included
- `config.json` — illustrative OpenLane-style configuration for the parameterized counter.
- `constraints.sdc` — example clock and I/O delay constraints.
- `flow_steps.md` — stages and evidence checklist.

## Important
This configuration is a starting template, not a guarantee that it runs unchanged on every
    OpenLane/OpenROAD version. Confirm configuration keys, PDK setup, top module, source paths and
    constraints for your installed flow. A real GDSII file can only be claimed after the flow runs
    successfully.

## Required evidence
Save actual logs/screenshots for synthesis, floorplan, placement, routing, timing/area status and
    final layout/GDSII if supported. Do not fabricate these results.
```

level3/task2_rtl_to_gdsii/config.json

```
{
  "DESIGN_NAME": "parameterized_counter",
  "VERILOG_FILES": [
    "dir::../../level2/task1_parameterized_rtl/rtl/parameterized_counter.v"
```

```
  ],
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 20,
  "FP_CORE_UTIL": 30,
  "PL_TARGET_DENSITY": 0.35,
  "DESIGN_IS_CORE": true
}
```

level3/task2_rtl_to_gdsii/constraints.sdc

```
# Example constraint template. Confirm port names and clock period for your chosen flow.
create_clock -name core_clock -period 20.0 [get_ports clk]
set_input_delay  1.0 -clock core_clock [get_ports {rst_n en}]
set_output_delay 1.0 -clock core_clock [all_outputs]
```

level3/task2_rtl_to_gdsii/flow_steps.md

```
# RTL-to-GDSII flow stages

1. **RTL checks and synthesis:** elaborate the design and map RTL into a gate-level netlist.
2. **Floorplan:** define core/die dimensions and place I/O and power structures as supported by the flow.
3. **Placement:** assign standard-cell locations and optimize placement.
4. **Clock-tree synthesis:** build a clock distribution network if required/enabled.
5. **Routing:** connect nets using available routing layers.
6. **Signoff-oriented checks:** inspect timing, design-rule and electrical reports supported by the
    chosen flow.
7. **GDSII export:** generate final layout database only if the configured flow and PDK support it.

Record tool versions, configuration, constraints, warnings/errors and the exact run directory. Do
    not commit licensed PDKs or private credentials.
```

level3/task3_timing_area_power/README.md

```
# Level 3 — Task 3: Timing, Area & Power Review

Use `implementation_review.md` to document real measurements from the synthesis or place-and-route
    flow. The template intentionally does not invent numerical results. Include report screenshots,
    a one-page implementation note, and the final checklist.
```

level3/task3_timing_area_power/implementation_review.md

```
# Timing, Area & Power Review — Fill with Measured Results

## Design and tool context
- Design/top module:
- RTL commit:
- Tool and version:
- Target platform / PDK (if applicable):
- Clock constraint:
- Run directory / report paths:

## Results table
| Metric | Measured value | Report / source |
|---|---|---|
| Cell area | TO BE MEASURED | |
| Critical-path slack (setup) | TO BE MEASURED | |
| Critical path startpoint / endpoint | TO BE MEASURED | |
| Power estimate (if supported) | TO BE MEASURED | |
| DRC / implementation status | TO BE MEASURED | |

## Critical-path review
- What is the longest path reported?
- Which cells/nets dominate delay?
- Is the clock constraint realistic and applied?
- Are reset paths treated as intended by the flow?

## One timing risk
Describe one measured risk and cite the report line or screenshot.

## One improvement to evaluate
State one proposed RTL/constraint/implementation change. Re-run the same flow and compare
    before/after results; do not claim an improvement without measured evidence.

## Checklist
- [ ] Clock and I/O constraints reviewed
- [ ] Area source report saved
- [ ] Critical path and slack recorded
- [ ] Power method/assumptions documented, if supported
- [ ] Warnings and unresolved issues listed
- [ ] Screenshots and tool versions included
```

Final Submission Checklist

- ■ Run each simulation and save command logs, VCD files and genuine GTKWave screenshots.
- ■ Run Yosys synthesis and retain the actual report and generated netlist.
- ■ Run OpenLane/OpenROAD where configured and retain real reports and layout evidence.
- ■ Complete timing/area/power review with measured values.
- ■ Push the repository and insert the actual repository URL.
- ■ Publish the required LinkedIn completion/project post and tag the official company handle provided by SAN Web Technology.
- ■ Submit/deposit work through the method provided by SAN Web Technology.
- ■ Remove passwords, private keys, secret tokens, licensed credentials and private IP.